

Chapitre 2 : Shell bash

TD/TP

Table des matières

1	Travaux Dirigés	2
1.1	Développements	2
1.1.1	Analyse.....	2
1.1.2	Construction	3
1.2	Redirections	3
1.2.1	Analyse.....	3
1.2.2	Construction	3
2	Travaux Pratiques	4
2.1	Etude de commandes	4
2.1.1	Filtres	4
2.1.2	Commandes head et tail	4
2.1.3	Commande wc	4
2.1.4	Commande grep.....	4
2.1.5	Commande cut	4
2.1.6	Commande sort	5
2.1.7	Commande tr	5
2.2	Exercices.....	5
2.2.1	infos.sh.....	5
2.2.2	projet.sh.....	5
2.2.3	comptes.sh	6
2.2.4	etc.sh	6
2.2.5	recents.sh	6
2.2.6	csv.sh	6
2.2.7	shells.sh.....	6
2.2.8	production.sh.....	6
2.2.9	nettoyage.sh.....	6
2.2.10	quibosse.sh	6
2.3	Remise	6

1 Travaux Dirigés

Les exercices suivants approfondissent quelques-unes des notions vues en cours. Un papier et un crayon suffisent pour les réaliser.

1.1 Développements

1.1.1 Analyse

Indiquez ce qu'affichent les commandes suivantes, en indiquant les développements que le shell effectue.

On suppose être dans le contexte suivant :

- La date - heure courante est 29/09/2025 11:17.53.
- L'utilisatrice courante est lamia, les seuls autres utilisateurs ayant un compte sur la machine sont root, alex et zoe.
- Le répertoire courant de lamia contient les entrées non cachées suivantes : rapport.odt main.c gui.c gui.h calc.c calc.h chap1.odt chap2.odt chap3.odt conclu.odt

```
1. $ echo utilitaires.{c,h}
2. $ echo ~root/log
3. $ echo *.odt
4. $ echo util{itaires.h,itaires.c}
5. $ echo *[0..9].?
6. $ echo {5..1} partez !
7. $ echo *.?
8. $ echo intro.{odt,pdf} chapitre{1..3}.odt conclu.{pdf,odt}
9. $ echo "*.c"
10. $ echo "Hello $(whoami) !"
11. $ echo ~maxime/projetS1
12. $ echo *pp*
13. $ echo a{1..3}b{1,2}
14. $ echo .???
15. $ echo ~zoe
16. $ echo a{0..9\}
17. $ echo \*
18. $ echo {0..9..2}
19. $ echo ~/projetS1
20. $ echo "je suis '$' (whoami) "
21. $ echo ~/${whoami}
22. $ echo {9..0..2}
23. $ echo a{1..3}b{1,2}
```

24. `$ echo ~zoe`
25. `$ echo 'Vous êtes dans $(pwd) '`
26. `$ echo $(date +%H):$(date +%M):$(date +%S)`

1.1.2 Construction

Indiquez la commande permettant d'afficher les informations suivantes.

1. Le message « Salut » suivi du répertoire courant de l'utilisateur, son répertoire personnel et son nom, séparés par des espaces
2. Les entrées du répertoire courant dont le suffixe du nom (i.e. partie suivant le `.`) est une lettre
3. Les chaînes `trucA.c trucB.c trucC.c ... trucP.c`
4. Les entrées du répertoire courant dont le nom est sur quatre caractères
5. Les entrées du répertoire courant dont le nom contient au moins deux caractères
6. Les entrées du répertoire courant dont le radical du nom contient les lettres `h` ou `c`
7. Les entrées du répertoire courant dont le nom contient les lettres `e`, `M` et `b`, dans cet ordre
8. Les entrées du répertoire courant dont le nom commence par `c` ou `A` et `a` au moins trois caractères
9. Les entrées du répertoire courant dont le nom se termine par `h` ou un chiffre
10. Les entrées du répertoire courant qui ont un suffixe et ne sont pas des sources `C`
11. Les entrées cachées du répertoire courant

1.2 Redirections

1.2.1 Analyse

Indiquez ce que font les commandes suivantes, en précisant à chaque fois combien d'arguments le shell passe à la commande et les fichiers éventuellement créés.

1. `$ cat /etc/passwd 2> foo.txt`
2. `$ cat < /etc/passwd 2> foo.txt`
3. `$ cat /etc/passwd > mdp.txt`
4. `$ cat /etc/passwd > mdp.txt 2>&1`
5. `$ cat /etc/group < /etc/passwd > mdp.txt`
6. `$ cat /etc/group > passwd > mdp.txt`
7. `$ cat /etc/group 2>&1 > mdp.txt`
8. `$ echo "je suis dans $(pwd)" > infos.txt`

1.2.2 Construction

Indiquez les commandes permettant de réaliser les actions suivantes.

1. Concaténer le fichier `/etc/group` au fichier `infos.txt`.
2. Afficher le message « erreur de syntaxe » sur la console via la sortie erreur.
3. Ajouter la ligne "Fin" au fichier `infos.txt`.

2 Travaux Pratiques

Les exercices suivants doivent être réalisés sur votre machine Linux, dans un shell. Comme précédemment, créez un répertoire propre à ce TP, dans lequel vous placerez votre production.

Le travail de ce TP est double : découvrir à l'aide du man les principales fonctionnalités de quelques commandes très utiles, développer ensuite des scripts qui mettent en œuvre ces fonctionnalités.

2.1 Etude de commandes

À l'aide du man, découvrez les commandes utilitaires `head`, `tail`, `wc`, `grep`, `cut`, `sort` et `tr` en répondant aux questions suivantes.

2.1.1 Filtres

Les commandes listées ci-dessus peuvent-elles être utilisées dans une conduite ? Autrement dit, est-ce que chacune de ces commandes propose une *forme* dans laquelle elle (a) prend les données à traiter sur son entrée standard (0), (b) affiche les données produites sur sa sortie standard (1), et (c) affiche ses messages d'erreur sur sa sortie erreur (2) ? Notez qu'une commande qui respecte ces critères est appelée un *filtre*.

Profitez de cette exploration pour vérifier si chacune de ces commandes propose une (autre) forme dans laquelle elle prend les données à traiter dans *plusieurs* fichiers qui lui sont passés en argument.

2.1.2 Commandes `head` et `tail`

S'agissant de `head`, quelles options permettent d'afficher les N premières lignes ? Toutes les lignes sauf les N dernières ? Transposez cette question à la commande `tail`.

2.1.3 Commande `wc`

Quelles sont les trois informations que la commande affiche par défaut ? Quelles options permettent de n'afficher qu'une seule des trois ? Quelle option permet d'afficher une synthèse de ces informations quand plusieurs fichiers sont passés en argument de la commande ?

2.1.4 Commande `grep`

La commande affiche les lignes d'un ou plusieurs fichiers correspondant à un motif (pattern).

Quelles options permettent de : ignorer la casse lors de la recherche ? Afficher les lignes qui *ne* correspondent *pas* au motif ? Afficher le nombre total des lignes qui correspondent au motif plutôt que les lignes elles-mêmes ? Préfixer chaque ligne correspondant au motif de son numéro dans le fichier ? Chercher de façon récursive dans les fichiers se trouvant dans un répertoire et ses sous-répertoires ? Afficher les N lignes qui précèdent et les M lignes qui suivent une ligne qui correspond au motif, pour les contextualiser ?

2.1.5 Commande `cut`

La commande extrait de chaque ligne un ensemble de champs, séparés par un délimiteur.

Quel est le délimiteur de champ par défaut de la commande ? Quelle option permet de le changer ? Quel est le numéro du premier champ, 0 ou 1 ? Quelle syntaxe permet d'afficher des champs consécutifs ? Des champs non consécutifs ?

2.1.6 Commande sort

La commande trie les lignes d'un ou plusieurs fichiers.

Quelle est le critère de tri par défaut de la commande ? Quelle option permet de spécifier la clé (key) du tri, c'est-à-dire le ou les champs selon lesquels les lignes seront triées ? Quelles options permettent de : ignorer la casse lors du tri ? Considérer la clé du tri comme un nombre et faire un tri numérique ? Inverser l'ordre de tri ? Ne retenir à l'issue du tri qu'un unique exemplaire de plusieurs lignes identiques, s'il y en a ?

2.1.7 Commande tr

La commande traduit des caractères en d'autres caractères.

Quelles options et arguments permettent de : changer les caractères a, b et c en 1, 2 et 3 respectivement ? Concaténer toutes les lignes d'un fichier, en les séparant par un espace ?

2.2 Exercices

Les paragraphes suivants décrivent les exercices à réaliser. Chaque exercice consiste à développer un script bash, dont le nom est le titre de l'exercice. Le paragraphe qui suit spécifie ce que le script doit faire. Les exercices sont classés par ordre croissant de difficulté¹.

Vos scripts seront exécutés au moyen de la commande bash, comme suit, en prenant comme exemple le script nommé `script.sh` :

```
$ bash script.sh
```

Notez que certains des exercices vous demanderont de créer vos propres jeux de test, par exemple en créant des fichiers avec du contenu.

2.2.1 infos.sh

En supposant que vous êtes l'utilisatrice lamia, que votre répertoire courant est /tmp et qu'il est 12 heures 32 minutes et 55 secondes, affiche très exactement la ligne suivante :

```
$HOME=/home/lamia, $PWD=/tmp, $(date)=12 :32.55
```

Le script s'adapte bien entendu au contexte courant (utilisateur courant, répertoire courant, date courante).

2.2.2 projet.sh

Crée dans le répertoire courant les répertoires et fichiers suivants : sous-répertoires nommés `projet_doc`, `projet_src`, `projet_test`, chacun de ces répertoires contenant les fichiers

¹ sachant que cette notion est subjective

nommés `fic01.txt`, `fic02.txt`... à `fic10.txt`. Le script crée l'arborescence en deux commandes exactement, une par ligne du script.

2.2.3 `comptes.sh`

Affiche sur une seule ligne la liste des utilisateurs ayant un compte sur la machine, liste triée par ordre alphabétique.

2.2.4 `etc.sh`

Liste dans le fichier nommé `etc.yy.mm.jj` (où `yy`, `mm` et `jj` représentent la date courante `jj/mm/yy`) le contenu du répertoire `/etc`, une entrée par ligne, et affiche le nombre de ces entrées.

2.2.5 `recents.sh`

Affiche les trois entrées du répertoire courant les plus récentes (date de dernière modification), triées par ordre alphabétique, une entrée par ligne.

2.2.6 `csv.sh`

Affiche la valeur du fichier CSV nommé `fichier.csv` qui est située à l'intersection de la ligne numéro `L` et de la colonne numéro `C`. Le script va chercher ces numéros (tous deux entiers) dans les fichiers `ligne.txt` et `colonne.txt`, respectivement. Les numéros sont valides, aucun test d'erreur n'est demandé.

Note : un fichier CSV est un fichier dont les valeurs sont séparées par un point-virgule ";". Il a l'extension `.csv` et peut être lu par Excel, qui le présente à l'écran sous forme de grille Excel.

2.2.7 `shells.sh`

Affiche, sur deux colonnes séparées par un espace, les utilisateurs ayant un compte sur la machine, suivi de leur shell de connexion, trié par ordre alphabétique des shells, et, pour les shells identiques, par ordre alphabétique des utilisateurs.

2.2.8 `production.sh`

Affiche la liste des fichiers `.c` et `.h` du répertoire courant, un par ligne, chacun suivi du nombre de ses lignes, le tout trié par nombres de lignes décroissants.

2.2.9 `nettoyage.sh`

Supprime du répertoire courant les fichiers modifiés il y a plus de trois jours, en demandant à chaque fois confirmation à l'utilisateur, et consigne dans le fichier `nettoyage.log` le nom des fichiers effectivement supprimés, un par ligne.

2.2.10 `quibosse.sh`

Affiche à l'écran les utilisateurs couramment connectés sur la machine (commande `who`), triés par ordre croissant d'heure de connexion, tout en les ajoutant à la fin du fichier `quibosse.log`. Vous pouvez vous aider de la commande `tee`.

2.3 Remise

Remettez svp vos fichiers `.sh` dans l'espace Moodle prévu à cet effet.

Vous devez les remettre à *plat*, c'est-à-dire sans répertoires ou sous-répertoires intermédiaires et sans les regrouper en archive, compressée ou non. En résumé, faites un simple glisser-déposer des différents fichiers depuis votre répertoire de développement dans l'espace Moodle.

Vous pouvez remettre vos fichiers autant de fois que vous le voulez jusqu'à la date limite de remise. Notez que toute nouvelle version doit *écraser* la précédente : un fichier dont le nom n'est pas correct, comme par exemple `arbo ( 1 ) . sh`, sera ignoré lors de la correction.

L'espace Moodle est configuré pour accepter les remises hors délai, mais notez s'il vous plait que des pénalités de retard s'appliquent dans ce cas.